CHARUSAT®

अमृतं तु विद्या

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical-7.1

Github Link: https://github.com/rudraaaajoshi/FDSA_BATCH_A/tree/main/Practical-7

Difficulty

Understanding front and rear movement was slightly challenging.

Error

Error occurs when the queue is full or empty.

Solution

Used a circular queue with front and rear pointers to add and serve tokens.

Outcome

Successfully implemented the fixed-capacity queue and reused freed spaces.

Code(Github) Link:

https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-7/pr-7.1.cpp

Output:

```
Token 1 joined the queue.
Token 2 joined the queue.
Token 3 joined the queue.
Queue: 1 2 3
Token 1 is served.
Queue: 2 3
Token 4 joined the queue.
Token 5 joined the queue.
Token 6 joined the queue.
Queue: 2 3 4 5 6
Front token: 2
```

Key Question:

Problem-1:

A simple array queue eventually reports being full even when slots have been freed at the front. Explain why this happens structurally — not by tracing an example, but by describing what the front and rear pointers represent and what their movement pattern causes over time?

Answer:

In a simple array queue, front moves forward when elements are served, while rear keeps moving forward when new elements are added. The freed spaces before front are not reused, so eventually rear reaches the end of the array and the queue reports full even though empty spaces exist at the front.

25DIT023

Practical-7.2

Difficulty

Easy. Understanding dynamic queue using linked nodes was slightly challenging.

Error

If attend is called when the queue is empty, it displays an empty queue error.

Solution

Used a linked list queue where new patients are added at the rear and served from the front.

Outcome

Successfully implemented an unlimited patient queue that can grow as needed and handles an empty queue safely.

Code(Github) Link:

https://github.com/rudraaaajoshi/FDSA_BATCH_A/blob/main/Practical-7/pr-7.2.cpp

Output:

```
Patient 101 arrived.
Current front patient: 101
Patient 102 arrived.
Current front patient: 101
Patient 103 arrived.
Current front patient: 101
Patient 101 attended.
Current front patient: 102
Patient 102 attended.
Current front patient: 103
Patient 104 arrived.
Current front patient: 103
```

Key Question:

Problem-2:

Your linked list queue has no capacity limit. If arrive operations vastly outnumber attend operations over a long period, what real consequence would this have on the system, and where in your code is that consequence invisible?

Answer:

If arrive operations are much more than attend, the queue will keep growing and may eventually use up available memory. This is invisible in the code because new Node keeps allocating memory without checking for a memory limit.